TP pas à pas : construire un data warehouse avec un ETL

Enseignante : Safa Chmengui

Présentation du TP

Objectif général. Comprendre, en le faisant soi-même, comment des données brutes et pleines d'erreurs deviennent un data warehouse fiable, interrogeable en SQL. On réalise un ETL complet (Extract, Transform, Load) en cinq étapes séparées, chacune dans son propre script, pour observer précisément ce que fait chaque étape.

À la fin du TP, vous saurez :

- charger des fichiers sources dans une zone de staging ;
- diagnostiquer la qualité des données avec des requêtes SQL ;
- écrire des règles de nettoyage et mesurer leur effet ;
- construire un schéma en étoile (table de faits et tables de dimensions) ;
- répondre à des questions métier avec des jointures.

Outils. Python 3.10 ou plus, la bibliothèque DuckDB (une base de données analytique qui tient dans un simple fichier), et PyCharm ou n'importe quel terminal.

Le scénario

Le *Café Jasmin*, à Tunis, vend des boissons, des viennoiseries et des plats, sur place et en livraison. Le gérant possède trois fichiers exportés de sa caisse et de son fichier clients. Il veut savoir combien il a gagné, quels jours, avec quels produits, dans quelles villes et par quel canal.

Problème : les fichiers contiennent des erreurs de saisie. Si on calcule directement dessus, les résultats seront faux. Votre mission est de construire un petit data warehouse propre pour répondre à ses questions.

Le déroulé

Étape	Script	Ce qu'on fait	Zone de données
Préparation	aucun	Étape 0 : installer Python et configurer l'interpréteur ; puis créer le dossier et les fichiers de données	`donnees/`
1. Extract	`etape1_extract.py`	Copier les fichiers tels quels	schéma `staging`
2. Contrôle qualité	`etape2_qualite.py`	Lister toutes les erreurs	lecture seule
3. Transform	`etape3_transform.py`	Corriger les erreurs, règle par règle	schéma `propre`

Étape	Script	Ce qu'on fait	Zone de données
4. Load	`etape4_load.py`	Construire le schéma en étoile	schéma `dwh`
5. Analyse BI	`etape5_bi.py`	Répondre aux questions du gérant	lecture seule

Toutes les étapes écrivent dans le même fichier, `mini_warehouse.duckdb`, qui représente le data warehouse. Chaque étape lit donc le résultat de la précédente : **il faut les exécuter dans l'ordre**.

Règle de travail pour tout le TP. À chaque étape : (1) lisez l'objectif, (2) recopiez ou téléchargez le script, (3) exécutez-le, (4) comparez avec le résultat attendu, (5) lisez l'explication, (6) répondez aux questions avant de passer à l'étape suivante.

Étape 0 : installer Python et configurer l'interpréteur du projet

Objectif. Avoir un projet PyCharm qui utilise son propre Python isolé (un *environnement virtuel*, ou `.venv`), avec DuckDB installé dedans. Cette configuration se fait une seule fois.

Pourquoi un interpréteur de projet ? L'interpréteur est le programme Python qui exécute vos scripts. Un environnement virtuel est une copie de Python réservée à un projet : les bibliothèques installées pour ce TP ne se mélangent pas avec celles d'autres projets.

0.1 Installer Python (si ce n'est pas déjà fait)

1. Ouvrir PowerShell et taper `python --version`.
2. Si la réponse est `Python 3.10` ou plus, passer directement à l'étape 0.2.
3. Sinon (message « Python was not found » ou commande non reconnue), installer Python :
    - avec winget : `winget install -e --id Python.Python.3.13` ;
    - ou depuis **python.org/downloads** : lancer l'installateur et **cocher impérativement « Add python.exe to PATH »** sur le premier écran, puis **Install Now**.
4. Désactiver le faux raccourci du Microsoft Store : **Paramètres → Applications → Paramètres d'applications avancés → Alias d'exécution d'application**, puis désactiver **python.exe** et **python3.exe**.
5. **Fermer puis rouvrir** PowerShell, et vérifier à nouveau avec `python --version`.

0.2 Créer ou ouvrir le projet dans PyCharm

- **Nouveau projet** : **File → New Project**. Dans *Location*, saisir `C:\Users\<votre_nom>\PycharmProjects\tp_lakehouse`. Dans la partie *Interpreter type*, choisir **Project venv** (ou **Virtualenv**), et dans *Python version* sélectionner le Python installé à l'étape 0.1. Décocher *Create a welcome script* (inutile pour le TP), puis cliquer sur **Create**. PyCharm crée le dossier `.venv` et configure l'interpréteur automatiquement : passer directement à l'étape 0.4.
- **Projet existant** (dossier déjà créé) : **File → Open**, choisir le dossier `tp_lakehouse`, puis cliquer sur **Trust Project**. Continuer à l'étape 0.3.

0.3 Configurer l'interpréteur d'un projet existant

1. Ouvrir **File → Settings** (raccourci `Ctrl + Alt + S`), puis **Project: tp_lakehouse → Python Interpreter**.
2. Cliquer sur **Add Interpreter** (en haut à droite), puis **Add Local Interpreter**.
3. Deux cas :
    - **le dossier `.venv` n'existe pas encore** : choisir **Generate new** (ou **New environment**), type **Virtualenv**, emplacement `tp_lakehouse\.venv`, interpréteur de base = le Python installé à l'étape 0.1 ;
    - **le dossier `.venv` existe déjà** (créé en ligne de commande avec `python -m venv .venv`) : choisir **Select existing**, type **Python**, et indiquer le chemin `C:\Users\<votre_nom>\PycharmProjects\tp_lakehouse\.venv\Scripts\python.exe`.
4. Cliquer sur **OK**, puis à nouveau sur **OK** pour fermer les paramètres.
5. **Vérification** : en bas à droite de la fenêtre PyCharm, la barre d'état doit afficher **.venv [3.13]** (ou votre version de Python).

0.4 Installer DuckDB dans l'interpréteur du projet

Deux méthodes, au choix :

- **Par le terminal intégré** : ouvrir **View → Tool Windows → Terminal** (`Alt + F12`). L'invite doit commencer par `(.venv)`, sinon fermer et rouvrir l'onglet du terminal. Puis taper :

```
pip install duckdb
```

- **Par l'interface** : **File → Settings → Project → Python Interpreter**, cliquer sur **+**, rechercher `duckdb`, puis **Install Package**.

0.5 Vérifier que tout fonctionne

Dans le terminal de PyCharm :

```
python -c "import sys, duckdb; print(sys.executable); print('DuckDB', duckdb.__version__)"
```

Résultat attendu. La première ligne affiche un chemin qui se termine par `tp_lakehouse\.venv\Scripts\python.exe`, et la seconde la version de DuckDB. Si le chemin ne contient pas `.venv`, le terminal n'utilise pas l'interpréteur du projet : reprendre l'étape 0.3.

0.6 Problèmes fréquents

Symptôme	Cause	Solution
`Python was not found`	Python non installé ou absent du PATH	Refaire l'étape 0.1 en cochant « Add python.exe to PATH »
`l'exécution de scripts est désactivée sur ce système`	Politique de sécurité de PowerShell	Taper `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, confirmer, puis relancer `.venv\Scripts\Activate.ps1`
`ModuleNotFoundError: No module named`	DuckDB installé dans	Vérifier l'interpréteur (étape 0.3), puis refaire

Symptôme	Cause	Solution
`'duckdb'`	un autre Python	`pip install duckdb` dans le terminal (`.venv`)
Le triangle vert ▶ lance `main.py`	La configuration d'exécution sélectionnée est « main »	Clic droit dans le script voulu → **Run '<nom du script>'**, ou supprimer `main.py`
`No files found that match the pattern "donnees/..."`	Le script est lancé depuis le mauvais dossier	**Run → Edit Configurations** → *Working directory* = le dossier `exercice_etl`

Préparation

P.1 Vérifier l'environnement

Une fois l'étape 0 terminée, ouvrir le projet `tp_lakehouse` dans PyCharm, puis le **Terminal** (en bas de la fenêtre). L'invite doit commencer par `(.venv)`. Sinon, activer l'environnement :

```
.venv\Scripts\Activate.ps1
```

Vérifier que DuckDB est installé :

```
python -c "import duckdb; print(duckdb.__version__)"
```

Si la commande renvoie une erreur, installer DuckDB avec `pip install duckdb`.

P.2 Créer le dossier de l'exercice

```
mkdir exercice_etl
mkdir exercice_etl\donnees
cd exercice_etl
```

Arborescence attendue à la fin du TP :

```
exercice_etl/
├── donnees/
│   ├── commandes.csv
│   ├── clients.csv
│   └── produits.csv
├── etape1_extract.py
├── etape2_qualite.py
├── etape3_transform.py
├── etape4_load.py
├── etape5_bi.py
└── mini_warehouse.duckdb      <- créé automatiquement à l'étape 1
```

Important. Toutes les commandes `python etapeX...` doivent être lancées depuis le dossier `exercice_etl`. Avec la méthode « clic droit → Run » de PyCharm, vérifier dans **Run → Edit Configurations** que le *Working directory* est bien `exercice_etl`.

P.3 Créer les fichiers de données

Créer les trois fichiers suivants dans `exercice_etl/donnees/` (dans PyCharm : clic droit sur `donnees` → **New → File**), puis coller le contenu exact. Ne corrigez surtout pas les erreurs : elles sont volontaires.

`donnees/commandes.csv` : les ventes enregistrées par la caisse.

```
commande_id,date_commande,client_id,produit_id,quantite,montant,canal
1,2025-03-01,1,1,2,7.000,sur place
2,2025-03-01,2,3,1,12.500,Livraison
3,02/03/2025,3,2,3,15.000,sur place
4,2025-03-02,1,4,1,4.500,SUR PLACE
5,2025-03-03,4,5,2,18.000,livraison
6,2025-03-03,2,1,1,3.500,sur place
6,2025-03-03,2,1,1,3.500,sur place
7,04/03/2025,5,3,2,25.000,livraison
8,2025-03-04,3,4,-2,9.000,sur place
9,2025-03-05,6,2,1,,sur place
10,2025-03-05,1,5,1,9.000,Livraison
11,06/03/2025,4,1,4,14.000,sur place
12,2025-03-06,5,4,2,9.000,sur place
12,2025-03-06,5,4,2,9.000,sur place
13,2025-03-07,6,3,1,12.500,livraison
14,2025-03-07,2,2,2,10.000,  sur place
15,2025-03-08,3,5,1,,livraison
16,2025-03-08,1,3,1,12.500,sur place
```

`donnees/clients.csv` : le fichier clients (attention, la ville du client 2 est `tunis` suivi d'un espace).

```
client_id,nom,ville,telephone
1,Amira Ben Ali,Tunis,20111222
2,Youssef Trabelsi,tunis ,22333444
3,Salma Gharbi,ARIANA,
4,Karim Jaziri,Ben Arous,25555666
5,Nour Hammami,,27777888
6,Mehdi Sassi,Ariana,29999000
```

`donnees/produits.csv` : la carte du café.

```
produit_id,nom_produit,categorie,prix_unitaire
1,Café express,Boisson,3.500
2,Croissant,Viennoiserie,5.000
3,Pizza margherita,Plat,12.500
4,Jus d'orange,Boisson,4.500
5,Salade tunisienne,Plat,9.000
```

P.4 Question d'observation (avant de coder)

Relire `commandes.csv` et `clients.csv` à l'œil nu. Lister **toutes les anomalies** que vous repérez. Gardez cette liste : vous la comparerez au diagnostic automatique de l'étape 2.

Étape 1 : Extract, charger les sources dans la zone de staging

Objectif

Copier les trois fichiers sources **tels quels** dans le data warehouse, dans une zone appelée *staging*. On ne corrige rien : on veut une photo fidèle de ce que les systèmes sources envoient.

Notion clé : la zone de staging

La zone de staging est une zone de transit. Elle sert à deux choses : ne lire les systèmes sources qu'une seule fois (on ne les surcharge pas), et garder une trace exacte des données reçues. Si une règle de nettoyage est fausse plus tard, on peut tout recalculer à partir du staging.

Démarche

1.1 Créer le fichier `etape1_extract.py` dans `exercice_etl` avec ce code :

```
"""
ÉTAPE 1 : EXTRACT
On copie les fichiers sources TELS QUELS dans la zone de staging du data warehouse.
Aucune correction : on veut une photo fidèle de ce que les systèmes sources nous
envoient.
"""
import duckdb

con = duckdb.connect("mini_warehouse.duckdb")   # crée (ou ouvre) le fichier du data
warehouse
con.sql("CREATE SCHEMA IF NOT EXISTS staging")

# all_varchar=true : toutes les colonnes sont lues comme du texte.
# On ne fait confiance à aucun type tant que les données n'ont pas été contrôlées.
con.sql("CREATE OR REPLACE TABLE staging.commandes AS "
        "SELECT * FROM read_csv('donnees/commandes.csv', all_varchar=true)")
con.sql("CREATE OR REPLACE TABLE staging.clients AS "
        "SELECT * FROM read_csv('donnees/clients.csv', all_varchar=true)")
con.sql("CREATE OR REPLACE TABLE staging.produits AS "
        "SELECT * FROM read_csv('donnees/produits.csv', all_varchar=true)")

print("=== staging.commandes (copie brute du fichier commandes.csv) ===")
con.sql("SELECT * FROM staging.commandes").show(max_rows=50)

print("=== staging.clients ===")
con.sql("SELECT * FROM staging.clients").show()

print("=== staging.produits ===")
con.sql("SELECT * FROM staging.produits").show()

for t in ["commandes", "clients", "produits"]:
    n = con.sql(f"SELECT count(*) FROM staging.{t}").fetchall()[0][0]
    print(f"staging.{t} : {n} lignes")

con.close()
print("\nÉtape 1 terminée. Observez les tableaux : repérez-vous déjà des erreurs ?")
```

1.2 Exécuter :

```
python etape1_extract.py
```
1.3 Vérifier qu'un fichier `mini_warehouse.duckdb` est apparu dans le dossier.

Résultat attendu

Les trois tables s'affichent en entier, puis :

```
staging.commandes : 18 lignes
staging.clients : 6 lignes
staging.produits : 5 lignes
```
Dans l'en-tête des tableaux, toutes les colonnes sont de type `varchar` (texte), et les montants vides apparaissent comme `NULL`.

Explication du code

Instruction	Rôle
`duckdb.connect("mini_warehouse.duckdb")`	Ouvre le data warehouse ; le fichier est créé s'il n'existe pas
`CREATE SCHEMA staging`	Crée un « tiroir » qui regroupe les tables brutes
`read_csv(..., all_varchar=true)`	Lit le CSV en gardant toutes les valeurs en texte, sans conversion
`CREATE OR REPLACE TABLE`	Recrée la table à chaque exécution : relancer le script ne crée pas de doublons
`.show(max_rows=50)`	Affiche le tableau dans la console
`con.close()`	Ferme proprement le fichier, pour que l'étape suivante puisse l'ouvrir

Questions

1. Pourquoi charge-t-on toutes les colonnes en texte (`all_varchar=true`) plutôt que de laisser DuckDB deviner les types ?
2. Dans `staging.commandes`, quelles lignes apparaissent deux fois ?
3. Lancez le script une deuxième fois. Le nombre de lignes change-t-il ? Pourquoi ?

Étape 2 : contrôle qualité, diagnostiquer les erreurs

Objectif

Trouver automatiquement, avec des requêtes SQL, **toutes les erreurs** présentes dans le staging, et les afficher ligne par ligne. Cette étape ne modifie rien : c'est un diagnostic, comme une prise de sang avant un traitement.

Notion clé : mesurer avant de corriger

On ne nettoie pas « au hasard ». On mesure d'abord chaque problème, ce qui permet de justifier chaque règle de nettoyage et de vérifier ensuite qu'elle a bien fonctionné.

Démarche

2.1 Créer le fichier `etape2_qualite.py` :

```
"""
ÉTAPE 2 : CONTRÔLE QUALITÉ
Avant de nettoyer, on cherche et on affiche chaque problème, ligne par ligne.
Rien n'est modifié dans cette étape : on fait seulement un diagnostic.
"""
import duckdb

con = duckdb.connect("mini_warehouse.duckdb")

print("=== Problème 1 : commandes en double (même commande_id plusieurs fois) ===")
con.sql("""
    SELECT commande_id, count(*) AS nb_fois
    FROM staging.commandes
    GROUP BY commande_id
    HAVING count(*) > 1
""").show()

print("=== Problème 2 : montants vides ===")
con.sql("SELECT * FROM staging.commandes WHERE montant IS NULL").show()

print("=== Problème 3 : quantités négatives (valeurs aberrantes) ===")
con.sql("SELECT * FROM staging.commandes WHERE quantite::INT < 0").show()

print("=== Problème 4 : dates au format jj/mm/aaaa au lieu de aaaa-mm-jj ===")
con.sql("SELECT commande_id, date_commande FROM staging.commandes WHERE date_commande
LIKE '%/%'").show()

print("=== Problème 5 : canal écrit de plusieurs façons (casse, espaces) ===")
con.sql("""
    SELECT '[' || canal || ']' AS canal_tel_quel, count(*) AS nb
    FROM staging.commandes GROUP BY canal ORDER BY canal
""").show()
print("   (les crochets [ ] montrent les espaces cachés)")

print("\n=== Problème 6 : villes mal écrites ou manquantes ===")
con.sql("SELECT client_id, nom, '[' || coalesce(ville, 'NULL') || ']' AS
ville_telle_quelle FROM staging.clients").show()

print("=== Résumé du diagnostic ===")
con.sql("""
    SELECT count(*)                                          AS total_lignes,
           count(*) - count(DISTINCT commande_id)            AS doublons,
           count(*) FILTER (WHERE montant IS NULL)           AS montants_vides,
           count(*) FILTER (WHERE quantite::INT < 0)         AS quantites_negatives,
           count(*) FILTER (WHERE date_commande LIKE '%/%')  AS dates_format_fr,
           count(DISTINCT canal)                             AS valeurs_de_canal
```

```
    FROM staging.commandes
""").show()

con.close()
print("Étape 2 terminée. On sait maintenant exactement ce qu'il faut corriger.")
```

2.2 Exécuter :

```
python etape2_qualite.py
```

Résultat attendu

Problème	Lignes concernées
1. Doublons	commandes 6 et 12 (chacune 2 fois)
2. Montants vides	commandes 9 et 15
3. Quantité négative	commande 8 (quantité −2)
4. Dates `jj/mm/aaaa`	commandes 3, 7 et 11
5. Canal	5 écritures : `[  sur place]`, `[Livraison]`, `[SUR PLACE]`, `[livraison]`, `[sur place]`
6. Villes	`[tunis ]` (client 2), `[ARIANA]` (client 3), `[NULL]` (client 5)

Résumé final : `18 | 2 | 2 | 1 | 3 | 5`.

Explication des requêtes

- **Doublons** : `GROUP BY commande_id` regroupe les lignes par numéro ; `HAVING count(*) > 1` ne garde que les numéros présents plusieurs fois.
- **Montants vides** : `WHERE montant IS NULL`. Un champ vide dans un CSV devient `NULL` (absence de valeur).
- **Quantités négatives** : `quantite::INT` convertit le texte en nombre pour pouvoir le comparer à 0.
- **Dates** : `LIKE '%/%'` cherche les dates qui contiennent une barre oblique.
- **Canal et villes** : on entoure la valeur de crochets avec `'[' || canal || ']'` pour rendre visibles les espaces au début ou à la fin, invisibles sinon.
- **Résumé** : `count(*) FILTER (WHERE ...)` compte uniquement les lignes qui respectent la condition. Les doublons se calculent en soustrayant le nombre d'identifiants distincts du nombre total de lignes.

Questions

1. Comparez ce diagnostic à la liste faite à l'œil nu en P.4. Quelles erreurs aviez-vous manquées ?
2. Pourquoi les espaces dans `[  sur place]` sont-ils dangereux pour un rapport ?
3. Le téléphone de la cliente 3 est vide. Faut-il supprimer cette cliente ? Justifiez.
4. Si on calculait le chiffre d'affaires directement sur le staging, quelles erreurs fausseraient le résultat ?

Étape 3 : Transform, nettoyer les données règle par règle

Objectif

Corriger chaque erreur trouvée à l'étape 2, en appliquant **une règle à la fois** et en comptant les lignes après chaque règle. Le résultat est rangé dans un nouveau schéma, `propre`. Le staging n'est jamais modifié.

Notion clé : les deux familles de règles

- **Rejeter** une ligne quand elle est inutilisable (doublon, montant manquant, valeur absurde). Le nombre de lignes diminue.
- **Corriger** une valeur quand on sait la réparer (format de date, majuscules, espaces). Le nombre de lignes ne change pas.

Démarche

3.1 Créer le fichier `etape3_transform.py` :

```
"""
ÉTAPE 3 : TRANSFORM (nettoyage)
On applique les règles de nettoyage UNE PAR UNE, en affichant le nombre de lignes
après chacune.
Le résultat est rangé dans le schéma "propre".
"""
import duckdb

con = duckdb.connect("mini_warehouse.duckdb")
con.sql("CREATE SCHEMA IF NOT EXISTS propre")

def compter(table):
    return con.sql(f"SELECT count(*) FROM {table}").fetchall()[0][0]

print("Départ :", compter("staging.commandes"), "lignes dans staging.commandes\n")

# --- Règle 1 : supprimer les doublons ---
con.sql("CREATE OR REPLACE TEMP TABLE r1 AS SELECT DISTINCT * FROM
staging.commandes")
print("Règle 1 - suppression des doublons          ->", compter("r1"), "lignes")

# --- Règle 2 : rejeter les commandes sans montant ---
con.sql("CREATE OR REPLACE TEMP TABLE r2 AS SELECT * FROM r1 WHERE montant IS NOT
NULL")
print("Règle 2 - rejet des montants vides          ->", compter("r2"), "lignes")

# --- Règle 3 : rejeter les quantités négatives ---
con.sql("CREATE OR REPLACE TEMP TABLE r3 AS SELECT * FROM r2 WHERE quantite::INT >
0")
print("Règle 3 - rejet des quantités négatives     ->", compter("r3"), "lignes")

# --- Règle 4 : une seule forme de date, et des vrais types ---
# try_strptime essaie un format et renvoie NULL s'il ne correspond pas ;
# coalesce garde le premier résultat non NULL.
con.sql("""
```

```
    CREATE OR REPLACE TEMP TABLE r4 AS
    SELECT commande_id::INT AS commande_id,
           coalesce(try_strptime(date_commande, '%Y-%m-%d'),
                    try_strptime(date_commande, '%d/%m/%Y'))::DATE AS date_commande,
           client_id::INT  AS client_id,
           produit_id::INT AS produit_id,
           quantite::INT   AS quantite,
           montant::DOUBLE AS montant,
           canal
    FROM r3
""")
print("Règle 4 - dates unifiées + types convertis  ->", compter("r4"), "lignes")

# --- Règle 5 : uniformiser le canal (espaces + minuscules) ---
con.sql("CREATE OR REPLACE TABLE propre.commandes AS "
        "SELECT * REPLACE (lower(trim(canal)) AS canal) FROM r4")
print("Règle 5 - canal en minuscules sans espaces  ->", compter("propre.commandes"),
"lignes")

print("\n=== propre.commandes (résultat final) ===")
con.sql("SELECT * FROM propre.commandes ORDER BY commande_id").show(max_rows=50)

# --- Nettoyage des clients : ville propre, 'Inconnue' si absente ---
# On met tout en minuscules, puis une majuscule au début de chaque mot : 'tunis ' ->
'Tunis'
con.sql("""
    CREATE OR REPLACE TABLE propre.clients AS
    SELECT client_id::INT AS client_id,
           nom,
           coalesce(array_to_string(list_transform(string_split(lower(trim(ville)), '
'),
                    lambda x: upper(x[1]) || x[2:]), ' '), 'Inconnue') AS ville
    FROM staging.clients
""")
print("=== propre.clients : avant / après ===")
con.sql("""
    SELECT s.client_id, '[' || coalesce(s.ville, 'NULL') || ']' AS ville_avant,
p.ville AS ville_apres
    FROM staging.clients s JOIN propre.clients p ON p.client_id = s.client_id::INT
    ORDER BY s.client_id::INT
""").show()

# --- Produits : déjà propres, on convertit seulement les types ---
con.sql("""
    CREATE OR REPLACE TABLE propre.produits AS
    SELECT produit_id::INT AS produit_id, nom_produit, categorie,
prix_unitaire::DOUBLE AS prix_unitaire
    FROM staging.produits
""")

con.close()
print("Étape 3 terminée. Les données sont propres et correctement typées.")
```

3.2 Exécuter :

```
python etape3_transform.py
```

Résultat attendu

```
Départ : 18 lignes dans staging.commandes

Règle 1 - suppression des doublons          -> 16 lignes
Règle 2 - rejet des montants vides          -> 14 lignes
Règle 3 - rejet des quantités négatives     -> 13 lignes
Règle 4 - dates unifiées + types convertis  -> 13 lignes
Règle 5 - canal en minuscules sans espaces  -> 13 lignes
```

Puis le tableau des 13 commandes propres : les dates sont toutes au format `2025-03-xx`, le canal ne vaut plus que `sur place` ou `livraison`, et les colonnes ont de vrais types (`int32`, `date`, `double`). Enfin, les villes : `[tunis ]` → `Tunis`, `[ARIANA]` → `Ariana`, `[NULL]` → `Inconnue`.

Explication des règles

Règle	Instruction SQL	Type	Effet
1	`SELECT DISTINCT *`	Rejet	Supprime les lignes strictement identiques : −2
2	`WHERE montant IS NOT NULL`	Rejet	Retire les commandes 9 et 15 : −2
3	`WHERE quantite::INT > 0`	Rejet	Retire la commande 8 : −1
4	`coalesce(try_strptime(...), try_strptime(...))`	Correction	Convertit les deux formats en une vraie date ; `::INT` et `::DOUBLE` donnent les bons types
5	`lower(trim(canal))`	Correction	`trim` enlève les espaces, `lower` met en minuscules
Ville	`upper(x[1]) || x[2:]` sur chaque mot, puis `coalesce(..., 'Inconnue')`	Correction	Majuscule au début de chaque mot ; une ville absente devient `Inconnue`

Les tables `r1` à `r4` sont temporaires (`TEMP`) : elles disparaissent à la fermeture. Seules `propre.commandes`, `propre.clients` et `propre.produits` sont enregistrées dans le data warehouse.

Questions

1. Pourquoi les règles 4 et 5 ne changent-elles pas le nombre de lignes ?
2. Que se passerait-il si on faisait la règle 4 (conversion de `quantite` en nombre) avant la règle 3 ? Et si une quantité contenait le texte `deux` ?
3. Pour la cliente 5 sans ville, on a choisi `Inconnue` plutôt que de la supprimer. Quel serait l'inconvénient de la supprimer ?
4. Proposez une autre règle utile qu'on pourrait ajouter (indice : comparer `montant` à `quantite × prix_unitaire`).

Étape 4 : Load, construire le schéma en étoile

Objectif

Organiser les données propres en **schéma en étoile** dans le schéma `dwh` : une table de faits au centre, entourée de trois tables de dimensions. C'est cette forme que les outils de BI interrogent.

Notion clé : faits et dimensions

```
                    dim_date
                    (QUAND)
                       |
  dim_client --- fact_commandes --- dim_produit
   (QUI)          (COMBIEN)          (QUOI)
```

- La **table de faits** (`fact_commandes`) contient les événements mesurables : une ligne par commande, avec des clés vers les dimensions et des mesures numériques (quantité, montant).
- Les **tables de dimensions** donnent le contexte : qui a acheté (`dim_client`), quoi (`dim_produit`), quand (`dim_date`). On s'en sert pour filtrer et regrouper.
- La **granularité** est le niveau de détail d'une ligne de faits. Ici : **une ligne = une commande**.

Démarche

4.1 Créer le fichier `etape4_load.py` :

```
"""
ÉTAPE 4 : LOAD (chargement en schéma en étoile)
On organise les données propres en 1 table de faits + 3 tables de dimensions, dans le
schéma "dwh".

                 dim_date
                    |
   dim_client --- fact_commandes --- dim_produit

Granularité de la table de faits : UNE LIGNE = UNE COMMANDE.
"""
import duckdb

con = duckdb.connect("mini_warehouse.duckdb")
con.sql("CREATE SCHEMA IF NOT EXISTS dwh")

# --- Dimension client : QUI a acheté ---
con.sql("CREATE OR REPLACE TABLE dwh.dim_client AS SELECT client_id, nom, ville FROM
propre.clients")

# --- Dimension produit : QUOI a été acheté ---
con.sql("CREATE OR REPLACE TABLE dwh.dim_produit AS "
        "SELECT produit_id, nom_produit, categorie, prix_unitaire FROM
propre.produits")

# --- Dimension date : QUAND (un calendrier, une ligne par jour du mois de mars 2025)
---
con.sql("""
    CREATE OR REPLACE TABLE dwh.dim_date AS
    SELECT d::DATE AS date_id,
```

```
           day(d)      AS jour,
           month(d)    AS mois,
           year(d)     AS annee,
           dayname(d)  AS jour_semaine
    FROM range(DATE '2025-03-01', DATE '2025-04-01', INTERVAL 1 DAY) t(d)
""")

# --- Table de faits : les commandes, avec les clés vers les dimensions et les
mesures ---
con.sql("""
    CREATE OR REPLACE TABLE dwh.fact_commandes AS
    SELECT commande_id,
           date_commande AS date_id,      -- clé vers dim_date
           client_id,                     -- clé vers dim_client
           produit_id,                    -- clé vers dim_produit
           canal,
           quantite,                      -- mesure
           montant AS montant_ht          -- mesure
    FROM propre.commandes
""")

for t in ["dim_client", "dim_produit", "dim_date", "fact_commandes"]:
    n = con.sql(f"SELECT count(*) FROM dwh.{t}").fetchall()[0][0]
    print(f"dwh.{t} : {n} lignes")

print("\n=== dwh.fact_commandes (centre de l'étoile) ===")
con.sql("SELECT * FROM dwh.fact_commandes ORDER BY commande_id").show(max_rows=50)

print("=== dwh.dim_client ===")
con.sql("SELECT * FROM dwh.dim_client").show()

print("=== dwh.dim_date (7 premiers jours) ===")
con.sql("SELECT * FROM dwh.dim_date LIMIT 7").show()

# --- Vérification : chaque commande doit trouver son client et son produit ---
orphelins = con.sql("""
    SELECT count(*) FROM dwh.fact_commandes f
    LEFT JOIN dwh.dim_client c USING (client_id)
    LEFT JOIN dwh.dim_produit p USING (produit_id)
    WHERE c.client_id IS NULL OR p.produit_id IS NULL
""").fetchall()[0][0]
print("Commandes sans client ou sans produit correspondant :", orphelins)

con.close()
print("Étape 4 terminée. Le data warehouse est prêt à être interrogé.")
```

4.2 Exécuter :

```
python etape4_load.py
```

Résultat attendu

```
dwh.dim_client : 6 lignes
dwh.dim_produit : 5 lignes
dwh.dim_date : 31 lignes
dwh.fact_commandes : 13 lignes
```

Puis la table de faits (13 commandes), la dimension client avec les villes propres, les 7 premiers jours du calendrier (`2025-03-01` est un `Saturday`), et enfin :

```
Commandes sans client ou sans produit correspondant : 0
```

Explication du code

- **`dim_client` et `dim_produit`** reprennent simplement les tables propres, en ne gardant que les colonnes utiles à l'analyse (le téléphone n'est pas repris : il ne sert à aucun rapport).
- **`dim_date`** est **générée**, pas lue dans un fichier : `range(DATE '2025-03-01', DATE '2025-04-01', INTERVAL 1 DAY)` produit les 31 jours de mars. Pour chaque jour, on calcule le jour, le mois, l'année et le nom du jour (`dayname`). Ainsi, on peut analyser par jour de la semaine sans recalcul à chaque requête.
- **`fact_commandes`** renomme `date_commande` en `date_id` pour bien montrer que c'est une **clé** vers `dim_date`, et `montant` en `montant_ht`.
- **La vérification finale** utilise des `LEFT JOIN` : si une commande pointait vers un client ou un produit inexistant, la colonne de droite serait `NULL`. Le résultat 0 prouve l'**intégrité référentielle** : chaque fait a ses dimensions.

Questions

1. Quelle est la granularité de `fact_commandes` ? Que faudrait-il changer pour avoir une ligne par jour ?
2. Pourquoi `dim_date` contient-elle 31 lignes alors qu'il n'y a des commandes que sur 8 jours ?
3. Dessinez le schéma en étoile sur papier, avec les colonnes de chaque table et les liens entre clés.
4. Pourquoi la colonne `canal` reste-t-elle dans la table de faits plutôt que dans une dimension ? Proposez une `dim_canal`.

Étape 5 : analyse BI, répondre aux questions du gérant

Objectif

Utiliser le data warehouse pour répondre aux questions métier. Chaque question revient à **joindre la table de faits à une ou plusieurs dimensions**, puis à regrouper et additionner.

Notion clé : le modèle d'une requête BI

Presque toutes les requêtes d'un schéma en étoile suivent le même modèle :

```
SELECT <attribut de la dimension>, sum(<mesure du fait>)
FROM fact_commandes f
JOIN <dimension> USING (<clé>)
GROUP BY <attribut de la dimension>
```

Démarche

5.1 Créer le fichier `etape5_bi.py` :

```
"""
ÉTAPE 5 : ANALYSE (BI)
On répond aux questions du gérant en joignant la table de faits aux dimensions.
```

```
"""
import duckdb

con = duckdb.connect("mini_warehouse.duckdb")

print("=== Question 1 : quel est le chiffre d'affaires total ? ===")
con.sql("""
    SELECT count(*) AS nb_commandes, round(sum(montant_ht), 3) AS ca_total
    FROM dwh.fact_commandes
""").show()

print("=== Question 2 : CA par jour de la semaine (jointure avec dim_date) ===")
con.sql("""
    SELECT d.jour_semaine, count(*) AS nb_commandes, round(sum(f.montant_ht), 3) AS
ca
    FROM dwh.fact_commandes f
    JOIN dwh.dim_date d USING (date_id)
    GROUP BY d.jour_semaine
    ORDER BY ca DESC
""").show()

print("=== Question 3 : CA par catégorie de produit (jointure avec dim_produit) ===")
con.sql("""
    SELECT p.categorie, sum(f.quantite) AS quantite_vendue, round(sum(f.montant_ht),
3) AS ca
    FROM dwh.fact_commandes f
    JOIN dwh.dim_produit p USING (produit_id)
    GROUP BY p.categorie
    ORDER BY ca DESC
""").show()

print("=== Question 4 : CA par ville du client (jointure avec dim_client) ===")
con.sql("""
    SELECT c.ville, count(*) AS nb_commandes, round(sum(f.montant_ht), 3) AS ca
    FROM dwh.fact_commandes f
    JOIN dwh.dim_client c USING (client_id)
    GROUP BY c.ville
    ORDER BY ca DESC
""").show()

print("=== Question 5 : sur place ou en livraison ? ===")
con.sql("""
    SELECT canal, count(*) AS nb_commandes, round(sum(montant_ht), 3) AS ca
    FROM dwh.fact_commandes
    GROUP BY canal
    ORDER BY ca DESC
""").show()

print("=== Question 6 : tout en même temps (les 3 dimensions) ===")
con.sql("""
    SELECT d.jour_semaine, c.ville, p.nom_produit, f.quantite, f.montant_ht
    FROM dwh.fact_commandes f
    JOIN dwh.dim_date d    USING (date_id)
    JOIN dwh.dim_client c  USING (client_id)
    JOIN dwh.dim_produit p USING (produit_id)
```

```
    ORDER BY f.montant_ht DESC
    LIMIT 5
""").show()

con.close()
print("Étape 5 terminée. Le cycle ETL complet est fini : sources -> staging -> propre
-> dwh -> BI.")
```

5.2 Exécuter :

```
python etape5_bi.py
```

Résultat attendu

Question	Réponse
1. CA total	**152,5 DT** pour 13 commandes
2. Meilleur jour	`Saturday` : 32,0 DT (3 commandes) ; le moins bon est `Wednesday` : 9,0 DT
3. Par catégorie	Plat 89,5 · Boisson 38,0 · Viennoiserie 25,0
4. Par ville	Tunis 59,0 · Inconnue 34,0 · Ben Arous 32,0 · Ariana 27,5
5. Par canal	livraison 77,0 (5 commandes) · sur place 75,5 (8 commandes)
6. Top 5	La plus grosse commande : 2 pizzas pour 25,0 DT, un mardi, cliente de ville inconnue

Explication

- **Question 1** n'a besoin d'aucune jointure : toutes les mesures sont dans la table de faits.
- **Questions 2, 3 et 4** suivent exactement le modèle ci-dessus, avec une dimension différente à chaque fois. `USING (date_id)` signifie « joindre sur la colonne `date_id`, présente dans les deux tables ».
- **Question 5** regroupe sur `canal`, qui est directement dans la table de faits.
- **Question 6** joint les trois dimensions à la fois : c'est toute la force du schéma en étoile, chaque commande retrouve son contexte complet (quand, qui, quoi).

L'effet du nettoyage, en chiffres

Si on avait calculé le CA directement sur le fichier brut, sans les étapes 2 à 4 :

- les commandes 6 et 12 auraient été comptées deux fois (+12,5 DT de CA fictif) ;
- la commande 8, avec une quantité négative, aurait compté 9 DT pour une vente impossible ;
- les canaux auraient été éclatés en 5 lignes au lieu de 2, et Tunis en 2 villes (`Tunis` et `tunis`).

Questions

1. Écrivez la requête qui donne le CA **par produit** (nom du produit), du plus vendu au moins vendu.
2. Écrivez la requête qui donne le CA par ville **et** par canal.
3. La ville `Inconnue` représente 34 DT. Quelle action recommanderiez-vous au gérant ?

4. Le gérant veut savoir si la livraison est rentable. Le panier moyen (CA ÷ nombre de commandes) est-il plus élevé en livraison ou sur place ?

Exercices de manipulation, compte rendu

Exercices de manipulation

Le meilleur moyen de comprendre un ETL est de le « casser » volontairement. Pour chaque exercice, modifiez le fichier indiqué, **relancez les cinq étapes dans l'ordre**, et notez ce qui change à chaque étape.

1. **Nouveau doublon.** Dans `commandes.csv`, copiez la ligne de la commande 1 à la fin du fichier. Quelle étape le détecte ? Lequel des chiffres de l'étape 3 change ?
2. **Nouvelle date française.** Ajoutez la ligne `17,15/03/2025,6,2,2,10.000,LIVRAISON`. Vérifiez qu'elle arrive bien dans `fact_commandes` avec la date `2025-03-15` et le canal `livraison`.
3. **Client inconnu.** Ajoutez la ligne `18,2025-03-09,99,1,1,3.500,sur place` (le client 99 n'existe pas). Que renvoie la vérification de l'étape 4 ? Le CA par ville de l'étape 5 change-t-il ? Pourquoi ?
4. **Nouvelle règle.** Dans `etape3_transform.py`, ajoutez une règle qui rejette les commandes dont le canal n'est ni `sur place` ni `livraison` après nettoyage. Testez-la en ajoutant une commande avec le canal `emporter`.

Compte rendu à rendre

- Les réponses aux questions de chaque étape (Préparation P.4, étapes 1 à 5).
- Une capture d'écran du résultat de chaque script.
- Pour chaque exercice de manipulation : ce que vous avez modifié, ce que vous avez observé, et votre explication.
- Un schéma en étoile dessiné (question 3 de l'étape 4).